CS 182 - HW 11 - Jie Wen

1. LoRA

(a)

If the model is **underfitting**, the **rank (k)** in LoRA should be increased.

- W is modified as $W_0 + AB$ with $A \in \mathbb{R}^{m\times k}, B \in \mathbb{R}^{k\times \ell}$.
- Small k means the update AB is low-capacity (low-rank), so it may be too weak to capture the new task.
- Increasing k increases the degrees of freedom in AB, making the effective adaptation more expressive and helping to reduce underfitting.

(Training LoRA on more layers or relaxing regularization could also be considered, but the primary LoRA-specific knob is **rank (k)**.)

(b)

If both A and B are **initialized to all zeros**, then initially

$$W = W_0 + AB = W_0 + 0 = W_0.$$

This is fine, but the problem lies with the **gradients**:

Let L be the loss. Then

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial W} \cdot B^\top, \quad \frac{\partial L}{\partial B} = A^\top \cdot \frac{\partial L}{\partial W}.$$

At initialization, $A = 0$ and $B = 0$, so

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial W} \cdot 0 = 0, \quad \frac{\partial L}{\partial B} = 0 \cdot \frac{\partial L}{\partial W} = 0.$$

With **zero gradients**, SGD-style updates will not change A or B at all, and LoRA will **never learn**—the model stays at the pretrained W_0 forever.

(c)

With the given initialization

```
A = torch.nn.Parameter(torch.empty(m, k))
B = torch.nn.Parameter(torch.empty(k, l))
torch.nn.init.xavier_uniform_(A)
torch.nn.init.xavier_uniform_(B)
```

both A and B are initialized with **standard Xavier-uniform scales**, as if they were full standalone layers.

But in LoRA, AB is added as a **small low-rank *perturbation*** on top of a well-trained W_0. Problems:

1. **Update magnitude too large at start**
   - Xavier-uniform gives entries of order $\mathcal{O}(1/\sqrt{m})$ and $\mathcal{O}(1/\sqrt{\ell})$.
   - The product AB can have non-negligible norm, so $W_0 + AB$ may initially differ quite a lot from W_0.
   - This can **destroy** the useful pretrained features and cause instability or degrade performance early in training.
2. **LoRA usually wants small, controlled updates**
   - In practice, LoRA often scales AB by a small factor (e.g. α/r) or initializes one of A, B near zero (e.g. A random, B zeros) so that the *initial* effective update is very small and training starts near the pretrained model.
   - With plain Xavier on both, this property is lost; the adaptation is not a gentle, low-rank tweak but a random large modification.

So, LoRA fine-tuning may not work well because the **initial perturbation AB is too large and unstructured**, leading to worse optimization and poorer use of the pretrained weights.

(d)

Parameters in A: $m \times k$
Parameters in B: $k \times \ell$

Total number of LoRA parameters:

$$N_{\text{LoRA}} = mk + k\ell = k(m + \ell).$$

With 4 bytes per float, the memory in **bytes** is

$$\text{Memory}_{\text{LoRA}} = 4 \cdot k(m+\ell) \text{ bytes}.$$

A single full-rank adaptation matrix for W (same size as W_0) would have $m \times \ell$ parameters, requiring

$$\text{Memory}_{\text{full}} = 4 \cdot m\ell \text{ bytes}.$$

Comparison:

- LoRA: $4k(m+\ell)$ bytes
- Full adaptation: $4m\ell$ bytes

Since typically $k \ll \min(m, \ell)$, we have

$$k(m+\ell) \ll m\ell,$$

so LoRA uses **much less memory** than storing a full adaptation matrix.

---

2. A Brief Introduction to Transformer Interpretability

(a) The Simplest Transformer (Zero-Layer Model)

(i) Expression for logits

Embedding:

$$\mathbf{X}^0 = \mathbf{W}_E \mathbf{T} \in \mathbb{R}^{d_{\text{model}} \times n_{\text{context}}}$$

Unembedding:

$$\mathbf{L} = \mathbf{W}_U \mathbf{X}^0 = \mathbf{W}_U \mathbf{W}_E \mathbf{T} \in \mathbb{R}^{n_{\text{vocab}} \times n_{\text{context}}}.$$

(ii) What algorithm is implemented?

This is just a single linear map from one-hot tokens to logits:

$$\mathbf{L}_{:,t} = \mathbf{W}_U \mathbf{W}_E \mathbf{T}_{:,t}.$$

So the prediction at position t depends **only** on the identity of the token at position t (its one-hot vector), and is independent of all other tokens in the sequence. Algorithmically it is equivalent to a multiclass linear classifier that maps each token to a fixed logit vector with no contextual information.

(b) Multi-Head Attention: Concatenation vs. Addition

Let $\mathbf{r}^h \in \mathbb{R}^{d_{\text{head}} \times n_{\text{context}}}$ be the value output of head h.

(i) Equivalence

Stacking the heads:

$$\mathbf{R}^H = \begin{bmatrix} \mathbf{r}^1 \\ \mathbf{r}^2 \\ \vdots \\ \mathbf{r}^H \end{bmatrix} \in \mathbb{R}^{(d_{\text{head}}H) \times n_{\text{context}}}.$$

Define the big output matrix by concatenating the per-head matrices horizontally:

$$\mathbf{W}_O^H = \begin{bmatrix} \mathbf{W}_O^1 & \mathbf{W}_O^2 & \cdots & \mathbf{W}_O^H \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times (d_{\text{head}}H)}.$$

Then

$$\mathbf{H} = \mathbf{W}_O^H \mathbf{R}^H = \begin{bmatrix} \mathbf{W}_O^1 & \cdots & \mathbf{W}_O^H \end{bmatrix} \begin{bmatrix} \mathbf{r}^1 \\ \vdots \\ \mathbf{r}^H \end{bmatrix} = \sum_{h=1}^{H} \mathbf{W}_O^h \mathbf{r}^h,$$

which is exactly the “additive and independent” formulation.

(ii) Pros / cons

- **Concatenation view:**
  - Advantage: matches the standard implementation as one large matrix multiply; easy to reason about dimensions ($d_{\text{model}} = Hd_{\text{head}}$) and efficient on hardware.
  - Disadvantage: conceptually obscures that each head is an independent circuit contributing additively to the residual stream.
- **Additive-independent view:**
  - Advantage: makes it clear that each head independently computes an update $\mathbf{W}_O^h \mathbf{r}^h$ and these updates simply sum in the residual stream, which is very convenient for interpretability.
  - Disadvantage: hides the fact that this can still be implemented as a single concatenated matrix multiply; slightly less faithful to typical code structure.

(c) The QK Circuit

Let $\mathbf{Q}_i = \mathbf{W}_Q \mathbf{X}_i$, $\mathbf{K}_j = \mathbf{W}_K \mathbf{X}_j$.

(i) Bilinear form and $\mathbf{W}_{QK}$

$$\begin{aligned}\mathbf{S}_{ij} &= \mathbf{Q}_i^\top \mathbf{K}_j = (\mathbf{W}_Q \mathbf{X}_i)^\top (\mathbf{W}_K \mathbf{X}_j) \\ &= \mathbf{X}_i^\top \mathbf{W}_Q^\top \mathbf{W}_K \mathbf{X}_j.\end{aligned}$$

Define

$$\mathbf{W}_{QK} := \mathbf{W}_Q^\top \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.$$

Then

$$\boxed{\mathbf{S}_{ij} = \mathbf{X}_i^\top \mathbf{W}_{QK} \mathbf{X}_j.}$$

(ii) Interpreting $\mathbf{W}_{QK}$ in $d_{\text{model}} = 3$

Write $\mathbf{X}_i = (x_{i1}, x_{i2}, x_{i3})^\top$, similarly for $\mathbf{X}_j$.

1. $\mathbf{W}_{QK} = \mathbf{I}$**:**

   $$\mathbf{S}_{ij} = \mathbf{X}_i^\top \mathbf{X}_j = x_{i1}x_{j1} + x_{i2}x_{j2} + x_{i3}x_{j3}.$$

   The score is high when $\mathbf{X}_i$ and $\mathbf{X}_j$ are similar vectors (large positive dot product).
   → The head "looks for tokens whose residual vectors are similar to the query token."
2. $\mathbf{W}_{QK} = \text{diag}(1, 0, 0)$**:**

   $$\mathbf{S}_{ij} = x_{i1}x_{j1}.$$

   Only the first coordinate matters; dimensions 2 and 3 are completely ignored in deciding attention.
   → The head "looks for tokens that match the query in the 1st feature," regardless of anything in the 2nd and 3rd features.
3. $\mathbf{W}_{QK} = \text{diag}(1, 1, -1)$**:**

   $$\mathbf{S}_{ij} = x_{i1}x_{j1} + x_{i2}x_{j2} - x_{i3}x_{j3}.$$

   The score is large when the first two coordinates match (same sign, large magnitude) **and** the third coordinate is opposite (so $-x_{i3}x_{j3}$ is positive and large).
   → The head "looks for tokens that are similar in the first two features but have an opposite third feature," unlike the identity case which prefers similarity in all three.
4. $\mathbf{W}_{QK} = -\mathbf{I}$**:**

   $$\mathbf{S}_{ij} = -\mathbf{X}_i^\top \mathbf{X}_j.$$

   This is large when $\mathbf{X}_i$ and $\mathbf{X}_j$ have a large negative dot product, i.e., are **anti-aligned**.
   After the softmax, tokens whose vectors point in the opposite direction to the query get the highest attention weights, while similar tokens (large positive $\mathbf{X}_i^\top \mathbf{X}_j$) receive very low attention.

(d) The OV Circuit

Recall:

$$h(\mathbf{X}^0) = \mathbf{W}_O^h \mathbf{W}_V^h \mathbf{X}^0 \mathbf{A}^{h\top} = \mathbf{W}_{OV}^h \mathbf{X}^0 \mathbf{A}^{h\top}.$$

(i) Attention-weighted average

Let $\mathbf{Y} = \mathbf{X}^0 \mathbf{A}^\top$. Its i -th column is

$$\mathbf{Y}_i = (\mathbf{X}^0 \mathbf{A}^\top)_{:,i} = \sum_{j=1}^{n_{\text{context}}} \mathbf{X}^0_{:,j} (\mathbf{A}^\top)_{j,i} = \sum_{j=1}^{n_{\text{context}}} \mathbf{X}^0_j \mathbf{A}_{ij}.$$

So

$$\boxed{(\mathbf{X}^0 \mathbf{A}^\top)_i = \sum_{j=1}^{n_{\text{context}}} \mathbf{A}_{ij} \mathbf{X}^0_j,}$$

an attention-weighted average (or linear combination) of source token vectors.

(ii) Final residual stream

With a single attention layer of H heads and no other components,

$$\mathbf{X}^{\text{final}} = \mathbf{X}^0 + \sum_{h=1}^{H} h(\mathbf{X}^0) = \mathbf{X}^0 + \sum_{h=1}^{H} \mathbf{W}_{OV}^h \mathbf{X}^0 \mathbf{A}^{h\top}.$$

(iii) Update vector lies in $\text{Col}(\mathbf{W}_{OV}^i)$

For head i,

$$h_i(\mathbf{X}^0) = \mathbf{W}_{OV}^i \mathbf{X}^0 \mathbf{A}^{i\top}.$$

Let $\mathbf{Y} = \mathbf{X}^0 \mathbf{A}^{i\top}$. The update at destination position t is

$$h_i(\mathbf{X}^0)_t = (\mathbf{W}_{OV}^i \mathbf{Y})_{:,t} = \mathbf{W}_{OV}^i \mathbf{Y}_t,$$

for some vector $\mathbf{Y}_t \in \mathbb{R}^{d_{\text{model}}}$.

Any vector of the form $\mathbf{W}_{OV}^i \mathbf{v}$ is a linear combination of the columns of $\mathbf{W}_{OV}^i$, hence lies in $\text{Col}(\mathbf{W}_{OV}^i)$. Therefore

$$\boxed{h_i(\mathbf{X}^0)_t \in \text{Col}(\mathbf{W}_{OV}^i).}$$

So head i can only *write* within this (typically low-dimensional) subspace.

(e) Read and Write Subspaces via SVD

(i) Rank bound

$$\mathbf{W}_{OV} = \mathbf{W}_O \mathbf{W}_V, \qquad \mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}},\ \mathbf{W}_V \in \mathbb{R}^{d_{\text{head}} \times d_{\text{model}}}.$$

For any matrices,

$$\text{rank}(\mathbf{AB}) \leq \min\big(\text{rank}(\mathbf{A}), \text{rank}(\mathbf{B})\big).$$

So

$$\text{rank}(\mathbf{W}_{OV}) \leq \min(d_{\text{head}}, d_{\text{head}}) = d_{\text{head}}.$$

In typical transformers $d_{\text{head}} \ll d_{\text{model}}$, hence $\mathbf{W}_{OV}$ has rank much smaller than d_{model}: it is a low-rank matrix.

(ii) Decomposition of $\mathbf{W}_{OV}\mathbf{X}_{\text{src}}$

Let the compact SVD be

$$\mathbf{W}_{OV} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

with $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{d_{\text{model}} \times r}$, $\mathbf{\Sigma} = \text{diag}(\sigma_1, \ldots, \sigma_r) \in \mathbb{R}^{r \times r}$, $r = \text{rank}(\mathbf{W}_{OV})$.

For $\mathbf{X}_{\text{src}} \in \mathbb{R}^{d_{\text{model}}}$,

$$\begin{aligned}
\mathbf{W}_{OV}\mathbf{X}_{\text{src}} &= \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \mathbf{X}_{\text{src}} \\
&= \mathbf{U}\mathbf{\Sigma} \begin{bmatrix} \mathbf{V}_1^\top \mathbf{X}_{\text{src}} \\ \vdots \\ \mathbf{V}_r^\top \mathbf{X}_{\text{src}} \end{bmatrix} \\
&= \sum_{k=1}^{r} \sigma_k (\mathbf{V}_k^\top \mathbf{X}_{\text{src}}) \mathbf{U}_k.
\end{aligned}$$

Thus

$$\boxed{\mathbf{W}_{OV}\mathbf{X}_{\text{src}} = \sum_{k=1}^{r} \sigma_k (\mathbf{V}_k^\top \mathbf{X}_{\text{src}}) \mathbf{U}_k.}$$

(iii) Read / write subspaces

- **(a) Read subspace.**
  The head first forms the coefficients $\mathbf{V}_k^\top \mathbf{X}_{\text{src}}$. Thus it can only "see" the component of $\mathbf{X}_{\text{src}}$ that lies in

  $$\boxed{\mathcal{R}_{\text{read}} = \text{span}(\mathbf{V}_1, \ldots, \mathbf{V}_r).}$$

- **(b) Write subspace.**
  The output is a linear combination of $\mathbf{U}_k$, so it always lies in

  $$\boxed{\mathcal{R}_{\text{write}} = \text{span}(\mathbf{U}_1, \ldots, \mathbf{U}_r).}$$

- **(c) Information orthogonal to the read subspace.**
  If $\mathbf{X}_{\text{src}}$ is orthogonal to all $\mathbf{V}_k$, then $\mathbf{V}_k^\top \mathbf{X}_{\text{src}} = 0$ for all k, so

  $$\mathbf{W}_{OV}\mathbf{X}_{\text{src}} = \mathbf{0}.$$

  Hence components of $\mathbf{X}_{\text{src}}$ orthogonal to the read subspace are completely ignored by this head—they produce no contribution to the output.

(iv) Connection to the value projection $\mathbf{W}_V$

The value vector is $\mathbf{v} = \mathbf{W}_V \mathbf{X}_{\text{src}} \in \mathbb{R}^{d_{\text{head}}}$. The read subspace $\text{span}(\mathbf{V}_1, \ldots, \mathbf{V}_r)$ coincides with the subspace of the residual stream that $\mathbf{W}_V$ is sensitive to (its row space). Any component of $\mathbf{X}_{\text{src}}$ in the nullspace of $\mathbf{W}_V$ is invisible and therefore lies outside the read subspace. Since $\mathbf{v}$ has dimension d_{head}, the head can encode at most d_{head} independent

directions, so it is natural that $r \leq d_{\text{head}}$.

(v) Synthesis: SVD as a description of information movement

The SVD

$$\mathbf{W}_{OV} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\top}$$

fully characterizes how an attention head moves information from one token's residual stream to another's. The right singular vectors $\mathbf{V}$ specify **which features** of the source token (read directions) are extracted; the singular values $\mathbf{\Sigma}$ scale how strongly each feature is transmitted; and the left singular vectors $\mathbf{U}$ specify **where in the destination residual stream** those features are written. The attention pattern $\mathbf{A}$ then determines *which* source tokens' transformed features (given by $\mathbf{W}_{OV}$) are routed to each destination position and with what weights.

---

3. Coding Question: Transformer Interpretability

---

4. Scaling Laws of Batch Size

(a) Linear regression + SGD

- **Curves (LR vs MSE for different batch sizes)**
  In the "Least-squares SGD: final test loss vs learning rate" plot, each batch size (2, 4, 16, 64, 256, 1024, 2048) has a **U-shaped** curve:
  - For very small learning rates, the final test MSE is high (under-training).
  - As the LR increases, the loss **decreases**, hits a flat "good" region, and then
  - Beyond some critical LR, the loss **blows up** (diverges and hits the clipped ceiling).
- **Batch size vs optimal LR**
  In the "batch size vs optimal learning rate" plot (log-log), the points form an **approximately straight line**: as batch size increases by ~2–4×, the optimal LR increases by a bit less than 2×. This looks like a **power law**

  $$\eta^* \propto B^{\alpha}$$

  with $\alpha \approx 0.4$–0.5 (sub-linear but clearly increasing).

  So the relationship **resembles a power-law / approximate linear-scaling rule**: larger batch → larger optimal LR, roughly $\eta^* \propto B^{1/2}$.

(b) MLP + SGD

- **Curves (LR vs MSE)**
  For the MLP, the "MLP + SGD: final test loss vs learning rate" plot looks very similar qualitatively:
  - U-shaped curves for each batch size.
  - There is again a broad "good" LR range before divergence.
- **Batch size vs optimal LR**
  In the "MLP + SGD: batch size vs optimal learning rate" plot, the optimal LR:

  - **Increases** from small batch sizes up to around $B \approx 256$,
  - Then **flattens or even slightly decreases** for the largest batches.

  So the scaling law is **weaker and less clean** than in the linear-regression case:

  - Still roughly power-law / increasing at small-to-medium batch sizes,
  - But with **saturation** and noise at large batch sizes, reflecting the more complex, non-convex optimization landscape of the MLP.

  In words: *for the MLP, LR still tends to grow with batch size, but the trend deviates more from a simple power law and has a weaker dependence for large batches.*

(c) MLP + Adam

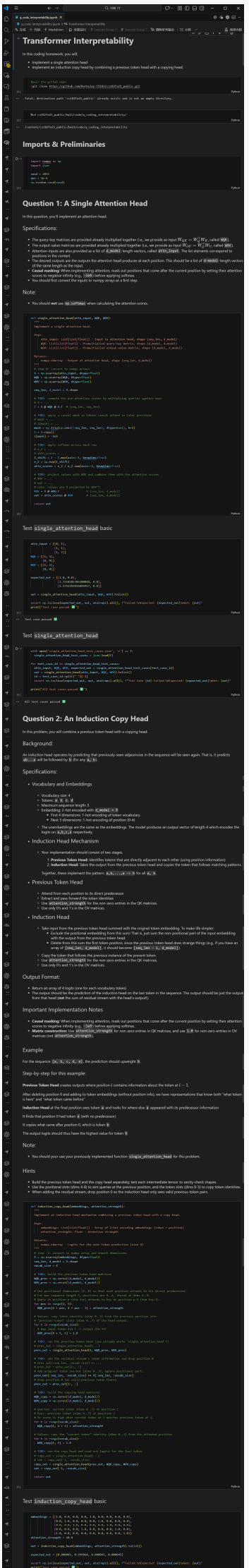
- **Curves (LR vs MSE)**
  The "MLP + Adam: final test loss vs learning rate" plot shows:
  - Much flatter curves at small LRs (Adam is more forgiving),
  - A rapid drop in MSE as LR goes from ~1e-4 to a few·1e-3,
  - Then a relatively broad good regime before performance starts to degrade at higher LRs.
- **Batch size vs optimal LR with Adam**
  In the "MLP + Adam: batch size vs optimal learning rate" plot:

  - The optimal LR is **much less sensitive** to batch size.
  - It starts around ~0.08 for $B = 2$, then **slowly decreases** to ~0.045 at $B \approx 1024$, with only a small bump at the largest batch.

  So with **Adam**, the LR scaling is **almost flat or even slightly *decreasing*** with batch size, very different from the roughly increasing power-law scaling seen with SGD.

---

5. Fermi Estimation for Large-scale Deep Learning Models



(b) Chinchilla Scaling Law Derivation

We need to minimize the loss function $L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_0$ subject to the constraint $C = C_0 N D$.
We can express D in terms of N using the constraint: $D = \frac{C}{C_0 N}$.
Substitute this into the loss equation (ignoring L_0 as it is constant):

$$L(N) = AN^{-\alpha} + B\left(\frac{C}{C_0 N}\right)^{-\beta} = AN^{-\alpha} + BC^{-\beta}C_0^{\beta}N^{\beta}$$

To find the minimum, we take the derivative with respect to N and set it to 0:

$$\frac{dL}{dN} = -\alpha A N^{-\alpha-1} + \beta B C^{-\beta} C_0^{\beta} N^{\beta-1} = 0$$

Rearranging the terms:

$$\alpha A N^{-\alpha-1} = \beta B C^{-\beta} C_0^{\beta} N^{\beta-1}$$

Multiply both sides by $N^{\alpha+1}$:

$$\alpha A = \beta B C^{-\beta} C_0^{\beta} N^{\alpha+\beta}$$

Solve for N_{opt}:

$$N^{\alpha+\beta} = \frac{\alpha A}{\beta B C_0^{\beta}} C^{\beta}$$

$$N_{opt} = \left(\frac{\alpha A}{\beta B C_0^{\beta}}\right)^{\frac{1}{\alpha+\beta}} C^{\frac{\beta}{\alpha+\beta}}$$

To find D_{opt}, we use the symmetry of the problem or substitution. Since $C = C_0 N D$, $N = \frac{C}{C_0 D}$. The loss balance condition $\frac{\alpha A}{N^\alpha} = \frac{\beta B}{D^\beta}$ implies:

$$D_{opt} = \left(\frac{\beta B}{\alpha A C_0^{\alpha}}\right)^{\frac{1}{\alpha+\beta}} C^{\frac{\alpha}{\alpha+\beta}}$$

Numerical Solution:
Given: $\alpha = 0.34, \beta = 0.28, A = 406.4, B = 410.7, C_0 = 6$.
Sum of exponents: $\alpha + \beta = 0.62$.
Ratio term $G = \frac{\alpha A}{\beta B} = \frac{0.34 \times 406.4}{0.28 \times 410.7} \approx \frac{138.18}{114.99} \approx 1.20$.

For N_{opt}:
Exponent: $\frac{\beta}{\alpha+\beta} = \frac{0.28}{0.62} \approx 0.45$.
Constant coefficient: $\left(\frac{1.20}{6^{0.28}}\right)^{\frac{1}{0.62}} \approx \left(\frac{1.20}{1.65}\right)^{1.61} \approx (0.727)^{1.61} \approx 0.60$.

$$N_{opt} \approx 0.60 \cdot C^{0.45}$$

For D_{opt}:
Exponent: $\frac{\alpha}{\alpha+\beta} = \frac{0.34}{0.62} \approx 0.55$.
Constant coefficient: $\left(\frac{1}{1.20 \cdot 6^{0.34}}\right)^{\frac{1}{0.62}} \approx \left(\frac{1}{1.20 \cdot 1.84}\right)^{1.61} \approx (0.45)^{1.61} \approx 0.27$.

$$D_{opt} \approx 0.27 \cdot C^{0.55}$$

(c) Compute and Dataset for GPT-6 (100T)

Using the direct statistical fitting provided:

$$N_{opt}(C) = 0.1C^{0.5} \quad \text{and} \quad D_{opt}(C) = 1.7C^{0.5}$$

We set $N = 10^{14}$ (100 trillion) and solve for C:

$$10^{14} = 0.1C^{0.5} \implies 10^{15} = C^{0.5} \implies C = (10^{15})^2 = 10^{30}$$

The compute cost is 10^{30} **FLOPs**.

Now we find the token count D:

$$D = 1.7C^{0.5} = 1.7(10^{30})^{0.5} = 1.7 \times 10^{15}$$

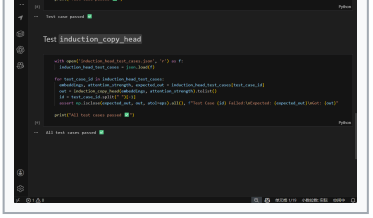
The training dataset would have 1.7×10^{15} **(1.7 quadrillion) tokens**.

(d) Dataset Size in Context

The question asks to evaluate **1000 trillion** (10^{15}) **tokens**.

Number of words:

$$10^{15} \text{ tokens} / 1.4 \text{ tokens/word} \approx 7.14 \times 10^{14} \text{ words}$$



Number of books:
One book ≈ 300 pages × 400 words/page = 120,000 words.

$$\text{Total Books} = \frac{7.14 \times 10^{14}}{1.2 \times 10^{5}} \approx 6 \times 10^{9}$$

That is **6 billion books**.

Comparison:
The Library of Congress holds approximately 170 million items (roughly 40 million cataloged books). Google Books has scanned roughly 25 to 40 million books.
Our calculated dataset (6 billion books) is roughly **150 times larger** than the entire estimated collection of Google Books or the book volume of the Library of Congress. It likely exceeds the total number of unique books ever written by humanity.

(e) Memory Requirement

1 Billion Parameter Model:

$$10^{9} \text{ params} \times 2 \text{ bytes (16-bit)} = 2 \times 10^{9} \text{ bytes} = 2 \text{ GB}$$

Hypothetical GPT-6 (1 Trillion Parameters):
Note: The prompt text in (e) asks for 1 trillion, despite the intro using 100T. I will calculate for 1T as requested here.

$$10^{12} \text{ params} \times 2 \text{ bytes} = 2 \times 10^{12} \text{ bytes} = 2 \text{ TB}$$

GPUs Required:
An H200 has 100 GB of VRAM.

$$\text{GPUs needed} = \frac{2000 \text{ GB}}{100 \text{ GB}} = 20 \text{ GPUs}$$

(f) Memory Cost

We are evaluating the cost for a **100 trillion (10^{14})** parameter model.
Model Size in MB: 10^{14} params $\times$ 2 bytes $= 200$ TB $= 200,000,000$ MB.

SSD Storage (Long-term):
Price: \$0.00005 / MB.

$$\text{Cost} = 200,000,000 \times 0.00005 = \$10,000$$

Since the SSD lasts 1 year, the cost is **\$10,000 per year**.

DRAM Memory:
Price: \$0.0040 / MB.

$$\text{Cost} = 200,000,000 \times 0.0040 = \$800,000$$

Assuming the hardware is purchased once, the capital cost is **\$800,000**.

(g) Memory Bandwidth and Latency

GPT-6 (100 Trillion Parameters):
Model size: 200 TB. Bandwidth: 4.8 TB/s.

$$\text{Latency} = \frac{200 \text{ TB}}{4.8 \text{ TB/s}} \approx 41.7 \text{ seconds}$$

Output per minute: $60/41.7 \approx$ **1.4 tokens per minute**.

GPT-3 (175 Billion Parameters):
Model size: $175 \times 10^{9} \times 2$ B ≈ 350 GB = 0.35 TB.

$$\text{Latency} = \frac{0.35 \text{ TB}}{4.8 \text{ TB/s}} \approx 0.073 \text{ seconds } (73 \text{ ms})$$

Output per minute: $60/0.073 \approx$ **822 tokens per minute**.

Batch Size Calculation for GPT-3:
We need to find the batch size where activation memory equals model memory (350 GB).
Model configuration: 96 layers, dimension $d = 12288$.
Activations per token per layer (assuming 16-bit precision): The "state" passed between layers is size d.
Total size per token = 96 layers $\times$ 12288 dim $\times$ 2 bytes ≈ 2.36 MB.
Note: This accounts for the residual stream state. If including KV Cache for attention, it would be higher, but using the forward pass state as a baseline:

$$\text{Batch Size} = \frac{350,000 \text{ MB}}{2.36 \text{ MB}} \approx 148,000$$

We need a batch size of approximately **148,000 tokens**.

(h) Training Cost

Compute Unit conversions:
1 PetaFLOP-day = 10^{15} FLOP/s $\times$ 86,400 s = 8.64×10^{19} FLOPs.
H200 Speed (100% util) = 1.98 PetaFLOP/s.

$$\text{Equivalent H200-hours} = \frac{1 \text{ day}}{1.98} \times 24 \text{ hours} \approx 12.1 \text{ hours}$$

Cost of 1 PetaFLOP-day = 12.1 hours $\times$ \$1.50/hr $\approx$ \$18.15.

GPT-3 Training Cost (Re-estimated with H200):
Compute: 3640 PetaFLOP-days.

$$\text{Cost} = 3640 \times \$18.15 \approx \$66,000$$

(This is hypothetically low because H200s are vastly more efficient than the V100/A100s used originally).

GPT-6 (100T) Training Cost:
Total Compute $C = 10^{30}$ FLOPs (from part c).

$$\text{in PetaFLOP-days} = \frac{10^{30}}{8.64 \times 10^{19}} \approx 1.16 \times 10^{10} \text{ PF-days}$$

$$\text{Cost} = 1.16 \times 10^{10} \times \$18.15 \approx \$210 \text{ Billion}$$

Comment on Cost:
At **\$210 Billion**, the cost exceeds the development of the iPhone (\$150M), the Three Gorges Dam (\$30B), and even the Manhattan Project (\$24B). It is comparable to the **Apollo Program (\$178B)**. This scale is likely beyond a single commercial actor like Google and sits squarely in the realm of a massive state actor or a consortium of the world's largest tech companies.

Wall Clock Time (30 million H200s):
Total throughput = $30 \times 10^6 \times 1.98 \times 10^{15}$ FLOP/s $\approx 5.94 \times 10^{22}$ FLOP/s.

$$\text{Time} = \frac{10^{30} \text{ FLOP}}{5.94 \times 10^{22} \text{ FLOP/s}} \approx 16,835 \text{ seconds}$$

$$\text{Hours} = \frac{16,835}{3600} \approx 4.7 \text{ hours}$$

(Wait, let me re-check the arithmetic. $10^{30}/6e22 = 1.6e7$*. My previous calculation was off by factor* 10^3*. Let's correct.)*
Corrected: $10^{30}/5.94 \times 10^{22} = 0.168 \times 10^8 = 16,835,016$ seconds.

$$\text{Hours} = \frac{16,835,016}{3600} \approx 4,676 \text{ hours} \approx 195 \text{ days}$$

(j) Inference Cost

GPT-5 (1 Trillion Parameters):
Cost to infer 1 token $\approx 2N = 2 \times 10^{12}$ FLOPs.
For 1 million tokens:

$$\text{Total FLOPs} = 10^6 \times 2 \times 10^{12} = 2 \times 10^{18} \text{ FLOPs}$$

Cost on H200:
H200 Speed: 1.98×10^{15} FLOP/s.
Time required: $\frac{2 \times 10^{18}}{1.98 \times 10^{15}} \approx 1010$ seconds ≈ 0.28 hours.

$$\text{Cost} = 0.28 \text{ hours} \times \$1.50/\text{hr} \approx \$0.42$$

Recouping \$1 Billion:
Revenue per 1M tokens = \$120. (Profit margin is high since cost is \$0.42).
Units to sell: $\frac{\$1,000,000,000}{\$120} \approx 8,333,333$ units.
Total tokens: 8.33 million $\times$ 1 million = 8.33 trillion tokens.

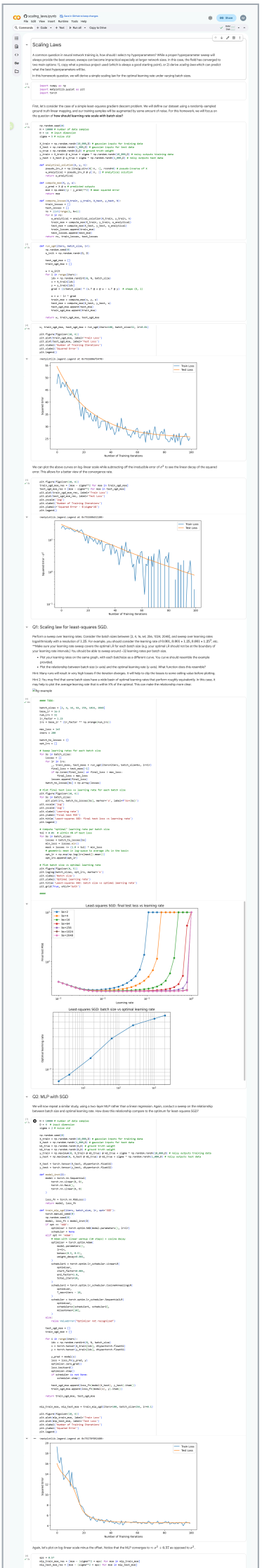
Equivalent Essays:
1 Essay = 1000 words $\approx$ 1400 tokens.

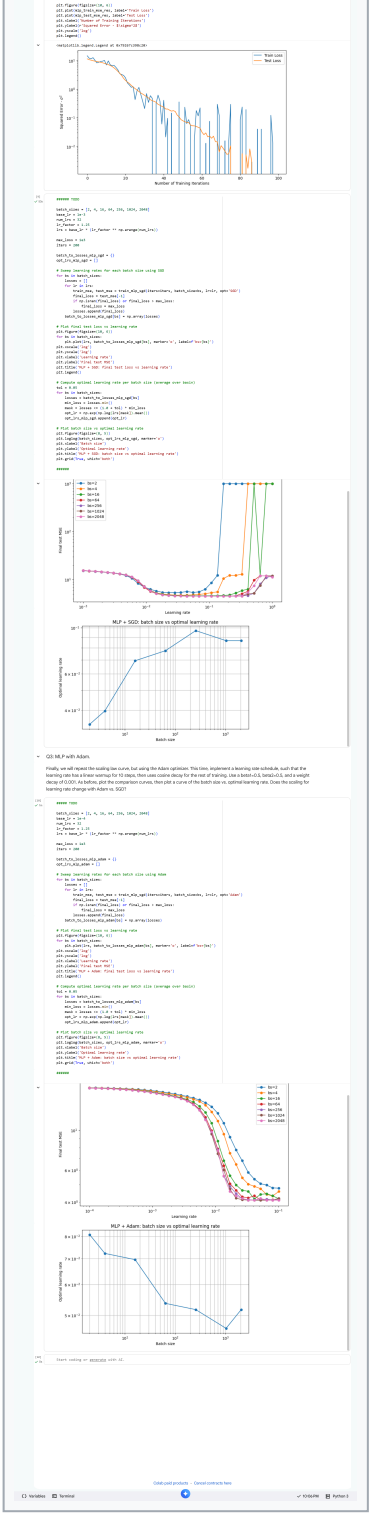
$$\text{Essays} = \frac{8.33 \times 10^{12}}{1400} \approx 5.95 \times 10^9 \text{ (approx 6 billion essays)}$$

(k) Energetic Cost

Minimal Energy per FLOP (Landauer):
A FLOP reduces 64 bits (two inputs) to 32 bits (one output), erasing 32 bits.

$$E_{min} = 32 \times k_B T \ln 2 = 32 \times (3 \times 10^{-21} \text{ J}) \approx 9.6 \times 10^{-20} \text{ J}$$





Minimal Power for H200:
Throughput: 1.98×10^{15} FLOP/s.

$$P_{min} = (1.98 \times 10^{15}) \times (9.6 \times 10^{-20}) \approx 1.9 \times 10^{-4} \text{ Watts}$$

Comparison:
Actual power is 700 Watts.

$$\text{Ratio} = \frac{700}{1.9 \times 10^{-4}} \approx 3.7 \times 10^{6}$$

Current technology uses over **3 million times** more energy than the fundamental physical limit.

(l) Environmental Cost

GPT-3 Carbon Cost:
Emissions: 552 tCO2. Cost: $112 / tCO2.

$$\text{Environmental Cost} = 552 \times 112 \approx \$61,824$$

Compared to the estimated $6 million training cost, the carbon tax would only add about **1%** to the total cost.

Burger Equivalence:
CO2 per burger (1/4 lb beef): $0.113 \text{ kg} \times 50 \text{ kg CO2/kg beef} = 5.65 \text{ kg CO2}$.

$$\text{Number of Burgers} = \frac{552,000 \text{ kg CO2}}{5.65} \approx 97,700 \text{ burgers}$$

6. Soft-Prompting Language Models

(a)

The training should be performed only on *real* tokens (question, reasoning, answer, newline), not on the soft-prompt vectors or padding.

- Input positions
  - 1–5: soft prompt (no ground-truth tokens)
  - 6–50: question tokens
  - 51–70: chain-of-thought tokens
  - 71: answer token
  - 72: newline
  - 73–100: padding

Because the target sequence is shifted by 1, the model's output at position t is trained to predict the target token at position $t + 1$. So:

- Outputs at positions **5–71** predict tokens at positions **6–72** (question + CoT + answer + newline).
- The following are **excluded**:
  - Outputs whose targets are the soft-prompt positions (targets 1–5).
  - Outputs whose targets are padding (targets 73–100), assuming padding is masked.

So: **use loss on outputs whose targets are tokens 6–72 (question, reasoning, answer, "\n"), ignore targets 1–5 and 73–100.**

(b)

All transformer weights (embeddings, attention, MLP, etc.) are **frozen**. The only trainable parameters are the soft-prompt vectors:

- 5 prompt vectors
- Each in $\mathbb{R}^E$ (embedding dimension E)

So number of trainable parameters is:

$$\boxed{5E}$$

(Independent of S, V, H, L, D.)

(c) True / False

(i) True.

In this setup the model weights are frozen and the prompt (hard or soft) is the same across all examples in a batch. In an autoregressive transformer, representations for positions 1–5 depend only on those positions (causal mask). So layer-wise representations for the prompt positions can be computed once per batch and reused for all examples in that batch, continuing the forward pass for the question tokens using those cached prefix states.

(ii) True.

A K-token hard prompt picks K *discrete* token embeddings from the vocabulary. A K-vector soft prompt directly chooses K arbitrary vectors in the same embedding space. Any hard prompt can be replicated by a soft prompt whose K vectors are exactly those token embeddings, so the set of possible soft prompts strictly contains the set of hard prompts. Therefore, **the best soft prompt can always match or outperform the best hard prompt** in principle.

(iii) False.

Even if computation is free, fully finetuning all model parameters is **not guaranteed** to be better. It gives more flexibility but can overfit, degrade performance on other tasks, or get stuck in worse local minima. Soft-prompt tuning is a restricted, more regularized adaptation and can sometimes generalize better than full finetuning; there is no theoretical guarantee that "more parameters ⇒ strictly better validation performance" for every task and dataset.

(iv) False.

In prompt tuning with a frozen backbone, each task's soft prompt is a separate set of parameters, and the base GPT weights never change. If a new soft prompt is tuned for Task A and only that prompt is prepended to Task A inputs, Task B's soft prompt and the shared LM weights remain unmodified, so Task B's performance does **not** suffer catastrophic forgetting.

(d) Meta-learning with soft prompts (MAML-style)

A family of related tasks exists, each to be solved by the *same frozen GPT* plus a *task-specific soft prompt*. A meta-learning approach allows for quickly obtaining a good soft prompt for a new task from a few labeled examples.

A natural MAML-style setup:

1. **Parameters to meta-learn:**
   Let θ be the **meta-initialization of the soft prompt** (e.g., K prompt vectors) shared across tasks. The GPT weights are frozen.

2. **Meta-training episodes (over training tasks):**
   For each task $\mathcal{T}$ in the task family:

   - Split its data into a **support set** ($D^{\mathcal{T}}_{\text{support}}$) and a **query set** ($D^{\mathcal{T}}_{\text{query}}$).
   - **Inner loop (task adaptation):**
     - Initialize task-specific prompt $p^{(0)}_{\mathcal{T}} = \theta$.
     - Take a small number of gradient steps on $D^{\mathcal{T}}_{\text{support}}$, updating only the soft prompt:

$$p^{(k+1)}_{\mathcal{T}} = p^{(k)}_{\mathcal{T}} - \alpha \nabla_{p_{\mathcal{T}}} \mathcal{L}_{\mathcal{T}}(p^{(k)}_{\mathcal{T}}; D^{\mathcal{T}}_{\text{support}})$$

     - After K steps, get adapted prompt $p'_{\mathcal{T}}$.
   - **Outer loop (meta-update):**
     - Evaluate loss on the query set:

$$\mathcal{L}^{\mathcal{T}}_{\text{meta}}(\theta) = \mathcal{L}_{\mathcal{T}}(p'_{\mathcal{T}}; D^{\mathcal{T}}_{\text{query}})$$

     - Backpropagate through the inner-loop updates to get $\nabla_\theta \mathcal{L}^{\mathcal{T}}_{\text{meta}}$ (standard MAML).
   - Aggregate over tasks and update θ with a meta learning rate β:

$$\theta \leftarrow \theta - \beta \sum_{\mathcal{T}} \nabla_\theta \mathcal{L}^{\mathcal{T}}_{\text{meta}}.$$

   Intuitively, this finds a *universal soft-prompt initialization* that can be quickly adapted to any task in the family using a few gradient steps on that task's data — exactly what MAML is designed to do.
3. **Meta-test (previously unseen task):**
   - For a new task $\mathcal{T}_{\text{new}}$, start with $p^{(0)}_{\mathcal{T}_{\text{new}}} = \theta$.
   - Run a few gradient steps on its small support set to get $p'_{\mathcal{T}_{\text{new}}}$.
   - Use GPT + $p'_{\mathcal{T}_{\text{new}}}$ to perform the task (e.g., generate chain-of-thought + answer).

Optionally, one can meta-learn **both** a shared "meta-prompt" and per-task residual prompts, or condition the soft prompt on a task description via a small "prompt encoder" and meta-learn its parameters; but the core idea is the same: **MAML over soft-prompt parameters with the backbone frozen.**

---

7. TinyML - Quantization and Pruning.

(a)

1. `fine_grained_prune`

```python
def fine_grained_prune(tensor: torch.Tensor, sparsity : float) -> torch.Tensor:
    """
    magnitude-based pruning for single tensor
    :param tensor: torch.(cuda.)Tensor, weight of conv/fc layer
    :param sparsity: float, pruning sparsity
        sparsity = #zeros / #elements = 1 - #nonzeros / #elements
    :return:
        torch.(cuda.)Tensor, mask for zeros
    """
    sparsity = min(max(0.0, sparsity), 1.0)
    if sparsity == 1.0:
        tensor.zero_()
        return torch.zeros_like(tensor)
    elif sparsity == 0.0:
        return torch.ones_like(tensor)

    num_elements = tensor.numel()

    ###################### YOUR CODE STARTS HERE ######################
    # Step 1: calculate the #zeros (please use round())
    num_zeros = round(sparsity * num_elements)

    # if rounding gives 0 zeros, just keep everything
    if num_zeros == 0:
        return torch.ones_like(tensor)

    # Step 2: calculate the importance of weight
    importance = tensor.abs()

    # Step 3: calculate the pruning threshold
    # kthvalue is 1-based; we want the num_zeros-th smallest magnitude
    flat_importance = importance.view(-1)
    k = min(max(1, num_zeros), num_elements)
    threshold, _ = torch.kthvalue(flat_importance, k)

    # Step 4: get binary mask (1 for nonzeros, 0 for zeros)
    mask = (importance > threshold).to(tensor.dtype)
    ###################### YOUR CODE ENDS HERE ######################
```

```
    # Step 5: apply mask to prune the tensor
    tensor.mul_(mask)

    return mask
```

2. Question 3 – `target_sparsity`

```
###################### YOUR CODE STARTS HERE ######################
target_sparsity = 0.6  # please modify the value of target_sparsity
###################### YOUR CODE ENDS HERE ######################
test_fine_grained_prune(target_sparsity=target_sparsity, target_nonzeros=10)
```

(25 elements total, 10 nonzeros ⇒ sparsity = 1 − 10/25 = 0.6.)

3. Channel pruning – `get_num_channels_to_keep` and pruning next conv input

```
def get_num_channels_to_keep(channels: int, prune_ratio: float) -> int:
    """A function to calculate the number of layers to PRESERVE after pruning
    Note that preserve_rate = 1. - prune_ratio
    """
    ###################### YOUR CODE STARTS HERE ######################
    preserve_rate = 1.0 - prune_ratio
    n_keep = int(round(channels * preserve_rate))
    # ensure at least 1 channel is kept
    return max(1, n_keep)
    ###################### YOUR CODE ENDS HERE ######################

@torch.no_grad()
def channel_prune(model: nn.Module,
                  prune_ratio: Union[List, float]) -> nn.Module:
    ...
        # prune the input of the next conv (hint: just one line of code)
        ###################### YOUR CODE STARTS HERE ######################
        next_conv.weight.set_(next_conv.weight.detach()[:, :n_keep, :, :])
        ###################### YOUR CODE ENDS HERE ######################

    return model
```

4. Channel importance – `get_input_channel_importance`

```
# function to sort the channels from important to non-important
def get_input_channel_importance(weight):
    in_channels = weight.shape[1]
    importances = []
    # compute the importance for each input channel
    for i_c in range(weight.shape[1]):
        channel_weight = weight.detach()[:, i_c]
        ###################### YOUR CODE STARTS HERE ######################
        # Frobenius norm of all weights connected to this input channel
        importance = torch.norm(channel_weight)
        ###################### YOUR CODE ENDS HERE ######################
        importances.append(importance.view(1))
    return torch.cat(importances)
```

5. Apply channel sorting – reordering next conv input

```
@torch.no_grad()
def apply_channel_sorting(model):
    model = copy.deepcopy(model)  # do not modify the original model
    # fetch all the conv and bn layers from the backbone
    all_convs = [m for m in model.backbone if isinstance(m, nn.Conv2d)]
    all_bns = [m for m in model.backbone if isinstance(m, nn.BatchNorm2d)]
    # iterate through conv layers
    for i_conv in range(len(all_convs) - 1):
        ...
        # sorting from large to small
        sort_idx = torch.argsort(importance, descending=True)

        # apply to previous conv and its following bn
        prev_conv.weight.copy_(torch.index_select(
            prev_conv.weight.detach(), 0, sort_idx))
        for tensor_name in ['weight', 'bias', 'running_mean', 'running_var']:
            tensor_to_apply = getattr(prev_bn, tensor_name)
            tensor_to_apply.copy_(
                torch.index_select(tensor_to_apply.detach(), 0, sort_idx)
            )
```

```
        # apply to the next conv input (hint: one line of code)
        ###################### YOUR CODE STARTS HERE ######################
        next_conv.weight.copy_(torch.index_select(
            next_conv.weight.detach(), 1, sort_idx
        ))
        ###################### YOUR CODE ENDS HERE ######################

    return model
```

i. **Common characteristics of the weight distributions**

Across almost all convolution layers, the weights look roughly **zero-mean and bell-shaped (approximately Gaussian)**:

- They are **symmetric around 0**.
- Most weights have **small magnitude**, clustered tightly near 0.
- Only a small fraction of weights lie in the **large-magnitude tails**.

The main exception is the classifier layer, which is still concentrated near small values but is a bit more skewed.

ii. **How these characteristics help pruning**

Because most weights have **very small magnitude**, magnitude-based pruning can safely remove many of them without changing the network's behavior much. Intuitively:

- Small-magnitude weights contribute little to the layer's output.
- The few large-magnitude weights (the tails) likely carry most of the "important" information and are preserved when we threshold on |W|.

So a zero-centered, heavy-concentration-near-0 distribution makes it easy to prune aggressively while keeping accuracy relatively high.

iii. **Relationship between pruning sparsity and accuracy**

From the sensitivity curves, as **sparsity increases** (i.e., we prune a larger fraction of weights), the **accuracy generally decreases**:

- At low to moderate sparsities (e.g., 0.4–0.6), many layers keep accuracy close to the dense model.
- At higher sparsities (around 0.7–0.9), the accuracy drops sharply, especially for some layers.

So the relationship is **monotonically decreasing**: more pruning → lower accuracy, with a "cliff" once we prune too aggressively.

iv. **Do all layers have the same sensitivity?**

No. Different layers show **different sensitivity**:

- Some layers (e.g., conv1, conv5, conv7, classifier) stay close to dense accuracy even at relatively high sparsity.
- Others (especially early/mid conv layers) show a much **steeper drop** in accuracy as sparsity increases.

So sensitivity is clearly **layer-dependent**.

v. **Most sensitive layer**

From the curves and printed numbers, `backbone.conv0.weight` is the most sensitive:

- Its accuracy collapses the fastest as sparsity increases (e.g., at 0.9 sparsity it drops to around 30%, much worse than other layers).

So the first convolutional layer is **the most sensitive to pruning sparsity** in this model.

(b)

`k_means_quantize`

```
from fast_pytorch_kmeans import KMeans

def k_means_quantize(fp32_tensor: torch.Tensor, bitwidth=4, codebook=None):
    """
    quantize tensor using k-means clustering
    :param fp32_tensor:
    :param bitwidth: [int] quantization bit width, default=4
    :param codebook: [Codebook] (the cluster centroids, the cluster label tensor)
    :return:
        [Codebook = (centroids, labels)]
            centroids: [torch.(cuda.)FloatTensor] the cluster centroids
            labels: [torch.(cuda.)LongTensor] cluster label tensor
    """
    if codebook is None:
        ################ YOUR CODE STARTS HERE ################
        # get number of clusters based on the quantization precision
```

```
        # hint: one line of code
        n_clusters = 1 << bitwidth
        ################ YOUR CODE ENDS HERE ##################
        # use k-means to get the quantization centroids
        kmeans = KMeans(n_clusters=n_clusters, mode='euclidean', verbose=0)
        labels = kmeans.fit_predict(fp32_tensor.view(-1, 1)).to(torch.long)
        centroids = kmeans.centroids.to(torch.float).view(-1)
        codebook = Codebook(centroids, labels)
    ################ YOUR CODE STARTS HERE ################
    # decode the codebook into k-means quantized tensor for inference
    # hint: one line of code
    quantized_tensor = codebook.centroids[codebook.labels]
    ################ YOUR CODE ENDS HERE ##################
    fp32_tensor.set_(quantized_tensor.view_as(fp32_tensor))
    return codebook
```

update_codebook

```
def update_codebook(fp32_tensor: torch.Tensor, codebook: Codebook):
    """
    update the centroids in the codebook using updated fp32_tensor
    :param fp32_tensor: [torch.(cuda.)Tensor]
    :param codebook: [Codebook] (the cluster centroids, the cluster label tensor)
    """
    n_clusters = codebook.centroids.numel()
    fp32_tensor = fp32_tensor.view(-1)
    for k in range(n_clusters):
    ################ YOUR CODE STARTS HERE ################
        # hint: one line of code
        mask = (codebook.labels == k)
        if mask.any():
            codebook.centroids[k] = fp32_tensor[mask].mean()
    ################ YOUR CODE ENDS HERE ##################
```

linear_quantize

```
def linear_quantize(fp_tensor, bitwidth, scale, zero_point, dtype=torch.int8) -> torch.Tensor:
    """
    linear quantization for single fp_tensor
      from
        fp_tensor = (quantized_tensor - zero_point) * scale
      we have,
        quantized_tensor = int(round(fp_tensor / scale)) + zero_point
    :param tensor: [torch.(cuda.)FloatTensor] floating tensor to be quantized
    :param bitwidth: [int] quantization bit width
    :param scale: [torch.(cuda.)FloatTensor] scaling factor
    :param zero_point: [torch.(cuda.)IntTensor] the desired centroid of tensor values
    :return:
        [torch.(cuda.)FloatTensor] quantized tensor whose values are integers
    """
    assert(fp_tensor.dtype == torch.float)
    assert(isinstance(scale, float) or
           (scale.dtype == torch.float and scale.dim() == fp_tensor.dim()))
    assert(isinstance(zero_point, int) or
           (zero_point.dtype == dtype and zero_point.dim() == fp_tensor.dim()))

    ################ YOUR CODE STARTS HERE ################
    # Step 1: scale the fp_tensor
    scaled_tensor = fp_tensor / scale
    # Step 2: round the floating value to integer value
    rounded_tensor = scaled_tensor.round()
    ################ YOUR CODE ENDS HERE ##################

    rounded_tensor = rounded_tensor.to(dtype)

    ################ YOUR CODE STARTS HERE ################
    # Step 3: shift the rounded_tensor to make zero_point 0
    if isinstance(zero_point, int):
        shifted_tensor = rounded_tensor + zero_point
    else:
        shifted_tensor = rounded_tensor + zero_point
    ################ YOUR CODE ENDS HERE ##################

    # Step 4: clamp the shifted_tensor to lie in bitwidth-bit range
    quantized_min, quantized_max = get_quantized_range(bitwidth)
    quantized_tensor = shifted_tensor.clamp_(quantized_min, quantized_max)
    return quantized_tensor
```

get_quantization_scale_and_zero_point

```
def get_quantization_scale_and_zero_point(fp_tensor, bitwidth):
    """
    get quantization scale for single tensor
    :param fp_tensor: [torch.(cuda.)Tensor] floating tensor to be quantized
    :param bitwidth: [int] quantization bit width
    :return:
        [float] scale
        [int] zero_point
    """
    quantized_min, quantized_max = get_quantized_range(bitwidth)
    fp_max = fp_tensor.max().item()
    fp_min = fp_tensor.min().item()

    ################ YOUR CODE STARTS HERE ################
    # hint: one line of code for calculating scale
    scale = (fp_max - fp_min) / (quantized_max - quantized_min)
    # hint: one line of code for calculating zero_point
    zero_point = quantized_min - fp_min / scale
    ################ YOUR CODE ENDS HERE ##################

    # clip the zero_point to fall in [quantized_min, quantized_max]
    if zero_point < quantized_min:
        zero_point = quantized_min
    elif zero_point > quantized_max:
        zero_point = quantized_max
    else: # convert from float to int using round()
        zero_point = round(zero_point)
    return scale, int(zero_point)
```

---

linear_quantize_bias_per_output_channel

```
def linear_quantize_bias_per_output_channel(bias, weight_scale, input_scale):
    """
    linear quantization for single bias tensor
        quantized_bias = fp_bias / bias_scale
    :param bias: [torch.FloatTensor] bias weight to be quantized
    :param weight_scale: [float or torch.FloatTensor] weight scale tensor
    :param input_scale: [float] input scale
    :return:
        [torch.IntTensor] quantized bias tensor
    """
    assert(bias.dim() == 1)
    assert(bias.dtype == torch.float)
    assert(isinstance(input_scale, float))
    if isinstance(weight_scale, torch.Tensor):
        assert(weight_scale.dtype == torch.float)
        weight_scale = weight_scale.view(-1)
        assert(bias.numel() == weight_scale.numel())

    ################ YOUR CODE STARTS HERE ################
    # hint: one line of code
    bias_scale = input_scale * weight_scale
    ################ YOUR CODE ENDS HERE ##################

    quantized_bias = linear_quantize(bias, 32, bias_scale,
                                     zero_point=0, dtype=torch.int32)
    return quantized_bias, bias_scale, 0
```

---

quantized_linear

```
def quantized_linear(input, weight, bias, feature_bitwidth, weight_bitwidth,
                     input_zero_point, output_zero_point,
                     input_scale, weight_scale, output_scale):
    """
    quantized fully-connected layer
    :param input: [torch.CharTensor] quantized input (torch.int8)
    :param weight: [torch.CharTensor] quantized weight (torch.int8)
    :param bias: [torch.IntTensor] shifted quantized bias or None (torch.int32)
    :param feature_bitwidth: [int] quantization bit width of input and output
    :param weight_bitwidth: [int] quantization bit width of weight
    :param input_zero_point: [int] input zero point
    :param output_zero_point: [int] output zero point
    :param input_scale: [float] input feature scale
    :param weight_scale: [torch.FloatTensor] weight per-channel scale
    :param output_scale: [float] output feature scale
    :return:
```

```
        [torch.CharIntTensor] quantized output feature (torch.int8)
    """
    assert(input.dtype == torch.int8)
    assert(weight.dtype == input.dtype)
    assert(bias is None or bias.dtype == torch.int32)
    assert(isinstance(input_zero_point, int))
    assert(isinstance(output_zero_point, int))
    assert(isinstance(input_scale, float))
    assert(isinstance(output_scale, float))
    assert(weight_scale.dtype == torch.float)

    # Step 1: integer-based fully-connected (8-bit multiplication with 32-bit accumulation)
    if 'cpu' in input.device.type:
        # use 32-b MAC for simplicity
        output = torch.nn.functional.linear(input.to(torch.int32), weight.to(torch.int32), bias)
    else:
        # current version pytorch does not yet support integer-based linear() on GPUs
        output = torch.nn.functional.linear(input.float(), weight.float(), bias.float())

    ################ YOUR CODE STARTS HERE ################
    # Step 2: scale the output
    #         hint: 1. scales are floating numbers, we need to convert output to float as well
    #               2. the shape of weight scale is [oc, 1, 1, 1] while the shape of output is [batch_size, oc]
    output = output.to(torch.float32)
    scale = (input_scale * weight_scale.view(-1)) / output_scale
    output = output * scale.view(1, -1)

    # Step 3: shift output by output_zero_point
    #         hint: one line of code
    output = output + output_zero_point
    ################ YOUR CODE ENDS HERE ##################

    # Make sure all value lies in the bitwidth-bit range
    output = output.round().clamp(*get_quantized_range(feature_bitwidth)).to(torch.int8)
    return output
```

quantized_conv2d

```
def quantized_conv2d(input, weight, bias, feature_bitwidth, weight_bitwidth,
                     input_zero_point, output_zero_point,
                     input_scale, weight_scale, output_scale,
                     stride, padding, dilation, groups):
    """
    quantized 2d convolution
    :param input: [torch.CharTensor] quantized input (torch.int8)
    :param weight: [torch.CharTensor] quantized weight (torch.int8)
    :param bias: [torch.IntTensor] shifted quantized bias or None (torch.int32)
    :param feature_bitwidth: [int] quantization bit width of input and output
    :param weight_bitwidth: [int] quantization bit width of weight
    :param input_zero_point: [int] input zero point
    :param output_zero_point: [int] output zero point
    :param input_scale: [float] input feature scale
    :param weight_scale: [torch.FloatTensor] weight per-channel scale
    :param output_scale: [float] output feature scale
    :return:
        [torch.(cuda.)CharTensor] quantized output feature
    """
    assert(len(padding) == 4)
    assert(input.dtype == torch.int8)
    assert(weight.dtype == input.dtype)
    assert(bias is None or bias.dtype == torch.int32)
    assert(isinstance(input_zero_point, int))
    assert(isinstance(output_zero_point, int))
    assert(isinstance(input_scale, float))
    assert(isinstance(output_scale, float))
    assert(weight_scale.dtype == torch.float)

    # Step 1: calculate integer-based 2d convolution (8-bit multiplication with 32-bit accumulation)
    input = torch.nn.functional.pad(input, padding, 'constant', input_zero_point)
    if 'cpu' in input.device.type:
        # use 32-b MAC for simplicity
        output = torch.nn.functional.conv2d(input.to(torch.int32), weight.to(torch.int32), None, stride, 0, dilation, groups)
    else:
        # current version pytorch does not yet support integer-based conv2d() on GPUs
        output = torch.nn.functional.conv2d(input.float(), weight.float(), None, stride, 0, dilation, groups)
        output = output.round().to(torch.int32)
    if bias is not None:
        output = output + bias.view(1, -1, 1, 1)

    ################ YOUR CODE STARTS HERE ################
    # hint: this code block should be the very similar to quantized_linear()

    # Step 2: scale the output
```

```
    #         hint: 1. scales are floating numbers, we need to convert output to float as well
    #               2. the shape of weight scale is [oc, 1, 1, 1] while the shape of output is [batch_size, oc, height, width]
    output = output.to(torch.float32)
    scale = (input_scale * weight_scale) / output_scale
    scale = scale.view(1, -1, 1, 1)
    output = output * scale

    # Step 3: shift output by output_zero_point
    #         hint: one line of code
    output = output + output_zero_point
    ################ YOUR CODE ENDS HERE ##################

    # Make sure all value lies in the bitwidth-bit range
    output = output.round().clamp(*get_quantized_range(feature_bitwidth)).to(torch.int8)
    return output
```

`extra_preprocess` (Question 9.1)

```
print(quantized_model)

def extra_preprocess(x):
    # hint: you need to convert the original fp32 input of range (0, 1)
    #  into int8 format of range (-128, 127)
    ################ YOUR CODE STARTS HERE ################
    q = x * 255.0 - 128.0
    return q.round().clamp(-128, 127).to(torch.int8)
    ################ YOUR CODE ENDS HERE ##################

int8_model_accuracy = evaluate(quantized_model, dataloader['test'],
                               extra_preprocess=[extra_preprocess])
print(f"int8 model has accuracy={int8_model_accuracy:.2f}%")
```

i. For a 4-bit k-means quantization, the number of clusters is (2^4 = 16). So there would be **16 unique colors** in the quantized tensor (one color per centroid/cluster).

ii. In general, for an (n)-bit k-means quantization, the number of clusters is (2^n). So there would be **(2^n) unique colors** in the quantized tensor.

iii. Low-precision quantization still works well because neural networks are highly over-parameterized and inherently noise-tolerant. Small perturbations of the weights usually do not change the output much, so replacing each weight with a nearby centroid only introduces small errors. K-means chooses centroids that approximate the original weight distribution as well as possible, and during quantization-aware training the model can further adapt its weights and centroids to compensate for the quantization error. As a result, even with 4-bit or 2-bit weights, the network can preserve almost the same decision boundaries and maintain high accuracy.

8. Homework Process and Study Group

(a)

Lecture Notes

(b)

None

(c)

1 day